

Agent Zoo — Technical Implementation Report

Engineering reference for the agents in `opposition-agents-playing-mtg`

Abstract

This document is the engineering counterpart to the scientific writeup *Opposition Agents Playing Magic: The Gathering* (`paper/opposition_agents_mtg.tex`). Where the paper focuses on *objectives*, this report focuses on *exactly what the code does*: class signatures, control flow, configuration tables, failure modes, and the structured reasoning traces each agent emits. All cited paths are workspace-relative.

Contents

1	Theoretical background (1 page)	1
2	The contract every agent satisfies	2
3	The lineup	2
4	Per-agent reference	2
4.1	RandomAgent	2
4.2	HeuristicAgent	2
4.3	KGHeuristicAgent	3
4.4	OllamaAgent (and LLMAgent alias)	3
4.5	ActiveInferenceAgent	3
4.6	WorldModelAgent	4
4.7	LLMFusionAgent	4
4.8	HumanAgent	4
5	Reasoning trace contract	5
6	Test pointers	5
7	Theoretical foundations (extended)	5
7.1	POMDPs and partial observability	5
7.2	World models as latent imagination	5
7.3	Free energy and active inference	6
7.4	Knowledge-graph grounding	6
7.5	LLM-as-policy under legal-action constraints	6
7.6	Self-play promotion as fixed-point iteration	6
8	Where to look next	6

1 Theoretical background (1 page)

The framework treats Magic: The Gathering (MTG) as a partially-observable Markov game. Each player’s perspective is a POMDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, T, \Omega, R, \gamma)$ with hidden state (opponent hands and library order). Concretely:

- $\mathcal{S}$: full `GameState` including all players’ zones, the stack, mana pools, life totals, and book-keeping flags. Conservatively $|\mathcal{S}| > 10^{800}$.
- $\mathcal{O}$: controlled-perspective filter from `src/engine/zones.py`; only the active player’s hand, library top, and revealed permanents are visible.
- $\mathcal{A}$: legal `Action` list assembled by `src/engine/priority_loop.py`, typically $|\mathcal{A}| \in [1, 60]$.
- T : the rules engine itself — deterministic given the random seed and the active player’s choices.
- R : terminal ± 1 on win/loss; agents may shape via Δlife , Δcards , Δtempo when training a value head.

Three families of policies. The agents implement:

1. *Reactive* (Random, Heuristic, KG-Heuristic, LLM): map $o_t \rightarrow a_t$ directly, no internal state.
2. *Model-based* (World-Model): learn $\hat{T}$ in latent space and pick $\arg \max_a \mathbb{E}_{\hat{T}}[V(s_{t+H})]$.
3. *Belief-driven* (Active-Inference, LLM-Fusion): maintain a belief b_t over hidden state and minimise expected free energy $G(\pi) = \underbrace{-\mathbb{E}_q[R]}_{\text{pragmatic}} + \underbrace{-\mathbb{E}_q[\text{KL}(b_{t+1}||b_t)]}_{\text{epistemic}}$.

A self-contained derivation is in §7. See also the corresponding section in the companion paper.

2 The contract every agent satisfies

Every agent class implements a single method:

```
1 class MTGAgent(Protocol):
2     name: str
3
4     async def decide_action(
5         self,
6         game_state: GameState,
7         legal_actions: list[Action],
8     ) -> Action: ...
```

Agents must *never* mutate `game_state`; only the rules engine in `src/engine/` is allowed to. An agent that returns an action not in `legal_actions` is treated as a forfeit by `src/orchestrator/priority_loop`.

Determinism: any RNG owned by an agent must be a `random.Random` instance constructed from the seed passed by the runner. This guarantees (`seed`, `decklists`, `agent_types`) replays bit-exactly.

3 The lineup

4 Per-agent reference

4.1 RandomAgent

File: `src/agents/random_agent.py`. **Purpose:** baseline / smoke test.

Agent	Module	Strategy
Random	random_agent.py	Uniform over legal actions.
Heuristic	heuristic_agent.py	Hand-tuned action priorities.
KG-Heuristic	kg_heuristic_agent.py	Heuristic + Neo4j combo lookup.
Ollama (LLM)	ollama_agent.py	Local LLM picks an action index.
Active-Inference	active_inference_agent.py	Bayesian belief, EFE minimisation.
World-Model	world_model_agent.py	V+M+C latent rollouts.
LLM-Fusion	llm_fusion_agent.py	Weighted vote of all four signals.
Human	human_agent.py	Stdin prompt, used for debugging.

```

1 def decide_action(self, game_state, legal_actions):
2     return self._rng.choice(legal_actions)

```

Configuration: seed: int (forwarded to random.Random). Failure modes: none. Deterministic given a seed.

4.2 HeuristicAgent

File: src/agents/heuristic_agent.py.

Decision flow.

- 1: scored $\leftarrow [(\text{priority}(a), -\text{tiebreak}(a), a) \text{ for } a \in \text{legal_actions}]$
- 2: scored.sort(descending)
- 3: return scored[0].a

Priority table (ACTION_PRIORITIES):

Action type	Priority
PLAY_LAND	100
CAST_SPELL	90
DECLARE_ATTACKERS	80
DECLARE_BLOCKERS	70
ACTIVATE_ABILITY	60
PASS_PRIORITY	1
CONCEDE	0

Tiebreaks: prefer cheaper spells, prefer creatures with higher power when attacking. Failure modes: never holds up mana for instants (known limitation).

4.3 KGHeuristicAgent

File: src/agents/kg_heuristic_agent.py.

Wraps HeuristicAgent but rescores CAST_SPELL actions using the Neo4j graph through src/knowledge/kg_query.py:score_card_for_state:

$$\text{kg_score}(c) = 10 \cdot \mathbf{1}[c \in \text{NearComboMissingPieces}(s)] + \sum_{p \in \text{permanents}(s)} \text{strength}(\text{KG-edge}(c, p)). \quad (1)$$

The combined score is prio(a) + $\lambda \cdot \text{kg_score}(c)$ with $\lambda=5$. Configuration: neo4j_uri, auth, enable_combo_lookup (bool). Failure mode: if Neo4j is unreachable, falls back silently to plain HeuristicAgent.

4.4 OllamaAgent (and LLMAgent alias)

File: `src/agents/ollama_agent.py`.

Builds a textual board state via `format_state_for_llm()`, sends a chat completion request to `http://localhost:11434/api/chat` (model `gemma4:e2b`, temperature 0.2, `keep_alive=30m`, timeout 120s), and parses an integer action index out of the reply.

```
1 prompt = (  
2     "You are an MTG expert. Choose the BEST action by index.\n"  
3     f"State:\n{format_state_for_llm(state)}\n"  
4     f"Legal actions:\n{enumerate_actions(legal_actions)}\n"  
5     "Respond with ONLY the index."  
6 )  
7 resp = requests.post(OLLAMA_URL, json={"model": MODEL, "messages": [...]}  
8 idx = int(re.search(r"\d+", resp.json()["message"]["content"]).group())  
9 return legal_actions[idx % len(legal_actions)]
```

Failure modes: timeout $\rightarrow$ fall back to HeuristicAgent; non-integer reply $\rightarrow$ fall back to RandomAgent over a small legal-action subset.

4.5 ActiveInferenceAgent

File: `src/agents/active_inference_agent.py`.

Maintains a Dirichlet belief over the opponent's hand composition and minimises expected free energy:

$$G(\pi) = -\mathbb{E}_{q(s'|\pi)}[R(s')] - \text{KL}[q(s'|\pi) \parallel b_t]. \quad (2)$$

The first term is the pragmatic value (favouring high-utility outcomes), the second is the epistemic value (favouring information gain). Belief update on each opponent action uses Bayes' rule with a card-likelihood table seeded from the decklist (see `docs/OPPONENT_MODELING_WITH_DECKLIST.md`).

4.6 WorldModelAgent

File: `src/agents/world_model_agent.py`. **Backbone:** `src/world_model/world_model.py`.

Components (V+M+C, after Ha & Schmidhuber).

$$\begin{aligned} z_t &= \text{Enc}_V(o_t) && \text{(perceptual encoder, 768-d)} \\ g_t &= \text{Enc}_{KG}(\mathcal{G}, s_t) && \text{(GraphSAGE, 128-d)} \\ (h_t, c_t) &= \text{LSTM}_M([z_t; g_t], (h_{t-1}, c_{t-1})) \\ \hat{r}_t &= V(h_t) && \text{(value head)} \\ \pi(a|h_t) &= \text{softmax}(C(h_t)) && \text{(controller)} \end{aligned}$$

Decision rule. For each candidate action $a \in \mathcal{A}$, roll out K trajectories of horizon H in latent space (no actual rules-engine simulation):

$$\text{score}(a) = \frac{1}{K} \sum_{k=1}^K V(h_{t+H}^{(k)} \mid a_t = a). \quad (3)$$

Defaults: $K = 8$, $H = 10$, sampling temperature 1.15.

Checkpoint loading. `WorldModel.load` now tolerates three checkpoint shapes (canonical, JEPA-only, and free-form), warning-and-continuing rather than raising. The agent factory (`src/agents/__init__.py:_make_world_model`) falls back to a default-initialised network if the checkpoint file does not exist, so the harness can be smoke-tested before training.

4.7 LLMFusionAgent

File: `src/agents/llm_fusion_agent.py`.

Combines four signals into a single ranking:

$$\text{score}(a) = \alpha_h s_{\text{heur}}(a) + \alpha_g s_{\text{KG}}(a) + \alpha_w s_{\text{WM}}(a) + \alpha_\ell \log p_{\text{LLM}}(a), \quad (4)$$

with default weights $(\alpha_h, \alpha_g, \alpha_w, \alpha_\ell) = (0.15, 0.15, 0.40, 0.30)$ from `FusionConfig`. When any signal exceeds the `obvious_threshold` (0.8), it short-circuits the vote to save an LLM call.

Algorithm 1 LLM-Fusion turn (simplified)

Require: state s_t , legal actions $\mathcal{A}$, weights α

- 1: $S_h \leftarrow \text{HeuristicAgent.score_all}(s_t, \mathcal{A})$
 - 2: $S_g \leftarrow \text{KG.score_all}(s_t, \mathcal{A})$
 - 3: **if** $\max_a S_h(a) > \tau_{\text{obvious}}$ **then return** $\arg \max_a S_h(a)$
 - 4: **end if**
 - 5: $S_w \leftarrow \text{WM.value_rollout}(s_t, \mathcal{A})$
 - 6: $\mathcal{A}^* \leftarrow \text{top-}N \text{ actions by } S_w$
 - 7: $S_\ell(a) \leftarrow \log p_{\text{LLM}}(a \mid \text{prompt}(s_t, \mathcal{A}^*))$
 - 8: **return** $\arg \max_{a \in \mathcal{A}^*} \alpha_h S_h(a) + \alpha_g S_g(a) + \alpha_w S_w(a) + \alpha_\ell S_\ell(a)$
-

4.8 HumanAgent

File: `src/agents/human_agent.py`. Renders the legal actions to stdout and reads a 1-based index. Used by `examples/play_logged_game.py` for hand-debugging pathological states.

5 Reasoning trace contract

Every agent except Random/Human emits a structured JSON record on each decision into `state.log_jsonl(...)`:

```
1 {
2   "turn": 7, "phase": "main1", "agent": "llm_fusion",
3   "chosen_action_idx": 3,
4   "scores": {"heuristic": [...], "kg": [...], "wm": [...], "llm": [...]},
5   "weights": {"heur": 0.15, "kg": 0.15, "wm": 0.40, "llm": 0.30},
6   "elapsed_ms": 412,
7   "explanation": "WM rollout favours T+8 board state with..."
8 }
```

Trace files land at `runs/<exp>/game_NNN.jsonl` alongside the human-readable `game_NNN.log`.

6 Test pointers

Smoke commands.

```
1 .\venv\Scripts\python.exe -m pytest tests/ -q
2 .\venv\Scripts\python.exe examples\play_edh_pod.py --max-turns 6 --seed 7 --model none
3 scripts\run_overnight_benchmarks.bat --quick
```

Agent	Test file
RandomAgent	tests/test_random_agent.py
HeuristicAgent	tests/test_heuristic_agent.py
KGHeuristicAgent	tests/test_kg_heuristic_agent.py
OllamaAgent (mocked)	tests/test_ollama_agent.py
ActiveInferenceAgent	tests/test_active_inference_agent.py
WorldModelAgent	tests/test_world_model_agent.py
LLMFusionAgent	tests/test_llm_fusion_agent.py

7 Theoretical foundations (extended)

7.1 POMDPs and partial observability

A finite-horizon POMDP is the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, \Omega, R, \gamma, b_0)$. The optimal value function over beliefs satisfies

$$V^*(b) = \max_a \left[\sum_s b(s) R(s, a) + \gamma \sum_o \Pr(o | b, a) V^*(b'_o) \right], \quad (5)$$

where b'_o is the Bayes update of b after action a and observation o . Exact solution is intractable for $|\mathcal{S}| > 10^{800}$; all our policies are approximations.

7.2 World models as latent imagination

Following Ha & Schmidhuber and Hafner et al. (Dreamer), we factor the agent as V (encoder), M (recurrent dynamics), and C (controller). Training the dynamics jointly minimises

$$\mathcal{L}_{\text{dyn}} = \sum_t \|\hat{z}_{t+1} - \text{sg}(z_{t+1})\|_2^2 + \beta \cdot \text{KL}(q(z_t | o_t) \| p(z_t | h_{t-1})), \quad (6)$$

where sg is stop-gradient. We follow the JEPA recipe of LeCun & Assran: predict in representation space, not in pixel space. The single tunable knob is β (`-jepa-beta`, default 1.0).

7.3 Free energy and active inference

Friston’s free-energy principle frames perception and action as joint inference. The variational free energy is

$$F[q] = \underbrace{\mathbb{E}_q[\log q(s) - \log p(o, s)]}_{\text{evidence bound}} = \text{KL}(q \| p(s|o)) - \log p(o). \quad (7)$$

Action selection minimises *expected* free energy, which decomposes as

$$G(\pi) = \underbrace{-\mathbb{E}_q[\log p(o | C)]}_{\text{pragmatic}} + \underbrace{-\mathbb{E}_q[\text{KL}(q(s | o, \pi) \| q(s | \pi))]}_{\text{epistemic}}. \quad (8)$$

The pragmatic term encodes goal-seeking; the epistemic term encodes information-gathering. Bayesian RL is recovered when the prior C is uniform, and pure exploration is recovered when rewards are flat.

7.4 Knowledge-graph grounding

Cards form an inductive graph $\mathcal{G} = (V, E)$ with edges for *combo*, *synergy*, *counters*, and *enables* (see `src/knowledge/`). GraphSAGE produces inductive node embeddings $\mathbf{h}_v \in \mathbb{R}^{128}$ that are concatenated with the perceptual latent z_t before the recurrent dynamics. This is the V+M+C model with an additional *graph* channel Enc_{KG} .

7.5 LLM-as-policy under legal-action constraints

The LLM never freely generates text. Given the prompt $\rho(s_t, \mathcal{A})$, we read off

$$\log p_{\text{LLM}}(a \mid s_t) = \text{LM}(\text{idx}(a) \mid \rho(s_t, \mathcal{A})) \quad (9)$$

and mask any index outside $\{0, \dots, |\mathcal{A}| - 1\}$. This is the standard “constrained decoding” trick and prevents illegal hallucinated moves.

7.6 Self-play promotion as fixed-point iteration

The iterative training loop in `scripts/train_pipeline.py` implements a champion-vs-challenger schema: a challenger trained on the latest replay buffer plays a best-of- N against the current champion; promotion happens iff the win rate exceeds 0.55. This is a stochastic-approximation fixed-point on the policy improvement operator and converges (in expectation) provided the dynamics model is locally accurate.

8 Where to look next

- Companion paper: `paper/opposition_agents_mtg.tex`.
- Single source of truth: `IMPLEMENTATION_PLAN.md`.
- Engine docs: `ARCHITECTURE.md`, `DEVELOPMENT.md`.
- World-model design: `docs/WORLD_MODEL_DESIGN.md`.
- Opponent modelling: `docs/OPPONENT_MODELING_WITH_DECKLIST.md`.